

# Bases de données avancées

## Gestion des exceptions



Pr. ELALAOUI Hasna

[h.elalaoui@uca.ac.ma](mailto:h.elalaoui@uca.ac.ma)

1

# Introduction



## C'est quoi une exception?

- Dans PL/SQL, les erreurs et les warnings sont appelés "exceptions".
- Une exception est une erreur qui survient durant une exécution.
- 2 types d'exception :
  - Interne
    - – exception oracle prédéfinie
    - – exception oracle non prédéfinie
  - Externe (exception définie par l'utilisateur)



## Exception interne

- Les **exceptions internes** sont générées par le moteur du système (division par zéro, connexion non établie, table inexistante, privilèges insuffisants, mémoire saturée, espace disque insuffisant, ...).
- Une erreur interne est produite quand un bloc PL/SQL viole une règle d'Oracle ou dépasse une limite dépendant du système d'exploitation.
- A chaque erreur ORACLE correspond un code SQL (SQLCODE)
- Par exemple:**
  - diviser un nombre par zéro, l'exception est appelée **ZERO\_DIVIDE**.
  - Si SELECT ne trouve pas d'enregistrement alors l'exception est: **NO\_DATA\_FOUND**.
- Dans la diapositive suivante, nous listons les 20 exceptions Oracle les plus générées



## Exception **externe**: Problème

- l'erreur NO\_DATA\_FOUND, n'est opérationnelle qu'avec l'instruction SELECT.
- Une mise à jour ou une suppression (UPDATE et DELETE) d'un enregistrement inexistant ne déclenche pas l'exception.
- Pour gérer ces cas d'erreurs, il faut utiliser un curseur implicite et une exception utilisateur appelée également **exception externe**.



## Exception externe

- PL/SQL possède une collection d'exceptions prédéfinies. Chacune a un nom.
- Ces exceptions sont automatiquement déclenchées par PL/SQL lorsque l'erreur correspondante est produite.
- En plus de ces exceptions, le programmeur peut aussi créer ses propres exceptions pour traiter les erreurs dans l'application.
- Comprendre comment gérer une exception est très important.



## Récapitulatif

| EXCEPTION                        | DESCRIPTION                                                          | TRAITEMENT                                                                                |
|----------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Erreur pré-définie du oracle     | Une des 20 erreurs qui arrivent le plus fréquemment en langage P/SQL | Ne pas la déclarer et laisser oracle serveur l'émettre implicitement                      |
| Erreur non pré-définie du oracle | Toutes les autres erreur standard d'oracle                           | La déclarer à l'intérieur de la section declare et laisser oracle l'émettre implicitement |
| Erreur définie par l'utilisateur | Une condition que le programmeur définit comme anomalie              | La déclarer à l'intérieur de la section declare et son émission est explicite             |

2

## **Gestion des exceptions**





# Intercepter les exceptions

- Le mot clé **EXCEPTION** débute la section de la gestion des exceptions
- Plusieurs exceptions sont permises (définies et prédéfinies)
- Une seule exception est exécutée avant de sortir d'un bloc
- **WHEN OTHERS** est la dernière clause
  - Intercepte toutes les exceptions non gérées dans la même section d'exception
  - On utilise le gestionnaire d'erreurs **OTHERS** et On le place en dernier lieu après tous les autres gestionnaires d'erreurs, sinon il interceptera toutes les exceptions mêmes celles qui sont prédéfinies.
- L'interception des exceptions diffère suivant le type d'exception à gérer



## Exception Oracle prédéfinie

| Nom de l'exception      | Numéro    | Commentaires                                                                                            |
|-------------------------|-----------|---------------------------------------------------------------------------------------------------------|
| ACCESS_INTO_NULL        | ORA-06530 | Affectation d'une valeur à un objet non initialisé.                                                     |
| CASE_NOT_FOUND          | ORA-06592 | Aucun des choix de la structure CASE sans ELSE n'est effectué.                                          |
| COLLECTION_IS_NULL      | ORA-06531 | Utilisation d'une méthode autre que EXISTS sur une collection (nested table ou varray) non initialisée. |
| CURSOR_ALREADY_OPEN     | ORA-06511 | Ouverture d'un curseur déjà ouvert.                                                                     |
| DUP_VAL_ON_INDEX        | ORA-00001 | Insertion d'une ligne en doublon (clé primaire).                                                        |
| INVALID_CURSOR          | ORA-01001 | Ouverture interdite sur un curseur.                                                                     |
| INVALID_NUMBER          | ORA-01722 | Échec d'une conversion d'une chaîne de caractères en NUMBER.                                            |
| LOGIN_DENIED            | ORA-01017 | Connexion incorrecte.                                                                                   |
| NO_DATA_FOUND           | ORA-01403 | Requête ne retournant aucun résultat.                                                                   |
| NOT_LOGGED_ON           | ORA-01012 | Connexion inexistante.                                                                                  |
| PROGRAM_ERROR           | ORA-06501 | Problème PL/SQL interne (invitation au contact du support...).                                          |
| ROWTYPE_MISMATCH        | ORA-06504 | Incompatibilité de types entre une variable externe et une variable PL/SQL.                             |
| SELF_IS_NULL            | ORA-30625 | Appel d'une méthode d'un type sur un objet NULL (extension objet).                                      |
| STORAGE_ERROR           | ORA-06500 | Dépassement de capacité mémoire.                                                                        |
| SUBSCRIPT_BEYOND_COUNT  | ORA-06533 | Référence à un indice incorrect d'une collection (nested table ou varray) ou variables de type TABLE.   |
| SUBSCRIPT_OUTSIDE_LIMIT | ORA-06532 |                                                                                                         |
| SYS_INVALID_ROWID       | ORA-01410 | Échec d'une conversion d'une chaîne de caractères en ROWID.                                             |
| TIMEOUT_ON_RESOURCE     | ORA-00051 | Dépassement du délai alloué à une ressource.                                                            |
| TOO_MANY_ROWS           | ORA-01422 | Requête retournant plusieurs lignes.                                                                    |
| VALUE_ERROR             | ORA-06502 | Erreur arithmétique (conversion, troncature, taille) d'un NUMBER.                                       |

Utiliser le nom standard à l'intérieur de la section Exception

Les noms des exceptions prédéfinies oracle sont regroupées dans ce tableau



## Exemple erreur Oracle

```
DECLARE
    v_sal emp.sal%type;
BEGIN
    SELECT sal INTO v_sal from emp;
EXCEPTION
    WHEN TOO_MANY_ROWS then ... ;
    -- gérer erreur trop de lignes
    WHEN NO_DATA_FOUND then ... ;
    -- gérer erreur pas de ligne
    WHEN OTHERS then ... ;
    -- gérer toutes les autres erreurs
END ;
```



## Exercice erreur Oracle

- On souhaite écrire un bloc PL/SQL qui demande à l'utilisateur de saisir le numéro d'un employé et affiche par la suite son nom et son salaire
- Ne pas oublier de gérer les exceptions internes à Oracle



## Exception Oracle non-prédéfinie

- On peut intercepter une erreur oracle non prédéfinie en la déclarant en préalable, ou en utilisant la commande **OTHERS**, L'exception déclarée est implicitement déclenchée

1. Déclarer le nom de l'exception oracle non-prédéfinie

```
nomException EXCEPTION;
```

2. Associer l'exception déclarée au code standard de l'erreur oracle en utilisant l'instruction

```
PRAGMA EXCEPTION_INIT(nomException, numéroErreurOracle);
```

3. Traiter l'exception ainsi déclarée dans la section **EXCEPTION**



## Exemple

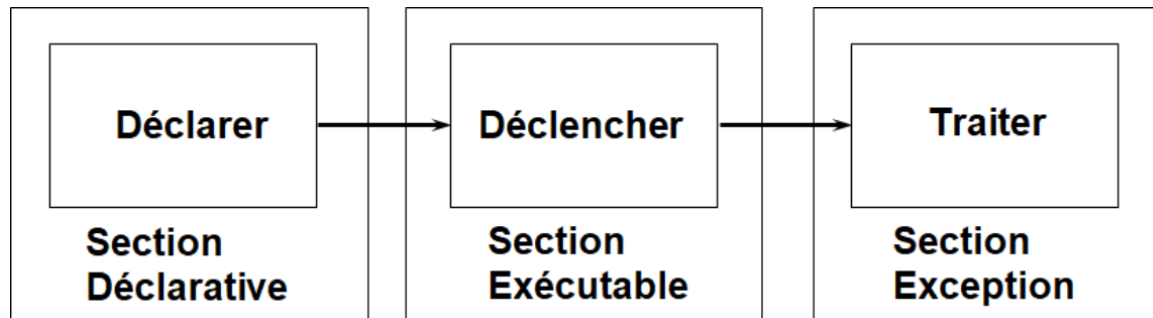
- Capturer l'erreur du Serveur Oracle numéro -2292 correspondant à la violation d'une contrainte d'intégrité.

```
DECLARE
e_emps    EXCEPTION;
PRAGMA EXCEPTION_INIT (e_products_remaining, -2292);
v_deptno dept.deptno%type:=&p_deptno
BEGIN
Delete from dept
Where deptno=v_deptno;
Commit;
EXCEPTION
WHEN e_emps THEN
dbms_output.put_line('Suppression impossible du dept : ' || to_char(v_deptno) ||
'employés existant ');
END;
```



# Exception utilisateur -1

- PL/SQL permet à l'utilisateur de définir ses propres exceptions.
- La gestion des anomalies utilisateur peut se faire dans un bloc PL/SQL en effectuant les opérations suivantes :
  1. Sont définies dans la section **DECLARE**
  2. Sont déclenchées explicitement dans la section **BEGIN** par l'instruction **RAISE**
  3. Dans la section **EXCEPTION**, référencer le nom défini dans la section **DECLARE**.





## Exception utilisateur -2

- Nommer l'erreur (type exception) dans la partie **DECLARE** du bloc
- Déterminer l'erreur et passer la main au traitement approprié par la commande **RAISE**.
- Effectuer le traitement défini dans la partie **EXCEPTION** du Bloc.
- On sort du bloc après l'exécution du traitement d'erreur.

```
DECLARE
    ...
    Nom_ano EXCEPTION;
BEGIN
    instructions ;
    IF (condition_anomalie) THEN RAISE Nom_ano;
    ...
EXCEPTION
    WHEN Nom_ano THEN (traitement);
END ;
```





## Exemple **exception utilisateur**

```
DECLARE
    salaire number;
    salaire_trop_bas EXCEPTION;
BEGIN
    SELECT sal INTO salaire FROM emp where empno= 50;
    if salaire < 300 THEN
        RAISE salaire_trop_bas;
    END IF;
    EXCEPTION
        WHEN salaire_trop_bas THEN
            dbms_output.put_line('Salaire trop bas');
        WHEN OTHERS THEN
            dbms_output.put_line(SQLERRM);
END;
```

3

## Fonctions d'interception des exceptions



## Aperçu général

- Obtenir des informations sur l'erreur – SQLCODE et SQLERRM
- Dans la section WHEN OTHERS du gestionnaire d'exception, on peut utiliser les fonctions **SQLCODE** et **SQLERRM** pour obtenir le numéro d'erreur et le message d'erreur respectivement.

### SQLCODE

- Renvoie la valeur numérique associé au code de l'erreur.
- On peut l'assigner à une variable de type NUMBER

### SQLERRM

- Renvoie le message associé au code de l'erreur.

- L'autre méthode consiste à associer une exception avec une erreur Oracle.



## Exemple **SQLCODE** et **SQLERRM**

```
declare
    newccode varchar2(5) := null;
begin
    update cours set ccode = newccode where ccode = 'c';
exception
    when dup_val_on_index then
        dbms_output.put_line('Duplication du code du cours ');
    when others then
        dbms_output.put_line( sqlcode);
        dbms_output.put_line( sqlerrm);
end;
```

- En exécutant ce programme, on obtient le message: **ORA-01407: impossible de mettre à jour ("SYSTEM"."COURS"."CCODE") avec NULL**
- Ce message est généré par WHEN OTHERS de la partie "gestion d'exception".
- Puisque (-01407) n'est associé à aucune exception prédéfinie, la partie WHEN OTHERS du gestionnaire d'exception est exécuté



## Procédure `RAISE_APPLICATION_ERROR`

- Permet de définir ces propres messages et codes d'erreurs
- Syntaxe : `RAISE_APPLICATION_ERROR(numerreur, message)`
- `numErreur` : valeur définie par l'utilisateur pour l'exception et comprise entre -20000 et -20999
- `message` : chaîne de caractère (max 2048 octets) décrivant l'erreur
- Peut être utilisé dans le code ou dans la section des exceptions



## Exemple

```
DECLARE
    salaire number;
BEGIN
    SELECT sal INTO salaire FROM emp where empno= 50;
    if salaire < 3000 THEN
        RAISE_APPLICATION_ERROR(-20001,'salaire bas');
    END IF;

    delete from emp where empno=5000;
    IF SQL%NOTFOUND then
        RAISE_APPLICATION_ERROR(-20002,'emp not exist!');
    END IF;
    EXCEPTION
        WHEN no_data_found then
            RAISE_APPLICATION_ERROR(-20002,'salarié n'existe pas');
        WHEN OTHERS THEN
            dbms_output.put_line(SQLERRM);
END;
/
```

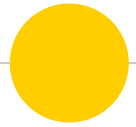


```
DECLARE
*
ERREUR ó la ligne 1 :
ORA-20002: salarié n'existe pas
ORA-06512: ó ligne 16
```



## Conclusion

- Une bonne gestion des exceptions n'est pas qu'une question technique – c'est ce qui fait la différence entre un programme amateur et un système professionnel.
- Toujours :
  - Anticiper les erreurs possibles
  - Informer l'utilisateur clairement
  - Maintenir l'intégrité des données
  - Logguer les erreurs pour le débogage
- Le code sans gestion d'erreurs est comme une voiture sans freins : ça marche... jusqu'au premier problème !



## **Fin du chapitre 4**